



中南大學

CENTRAL SOUTH UNIVERSITY

计算机视觉前沿技术

CSE

计算机学院

LLM常用强化学习方法

主讲： 沈成超

邮箱： scc.cs@csu.edu.cn

2026.05.15

目录



1. 课程引言
2. 主流方法
3. 工程设计
4. 发展趋势
5. 课程总结

1. 引言：为什么LLM需要强化学习

大语言模型最初通常通过自回归语言建模进行预训练，即最小化下一个token的预测误差：

$$\max_{\theta} \sum_t \log \pi_{\theta}(x_t | x_{< t})$$

这种方式擅长学习语言分布，但存在几个问题：

- 训练目标与真实需求不一致
- 无法直接优化非可微目标
- 需要长程决策能力

1. 引言：LLM强化学习的基本建模方式

把文本生成看成序列决策

LLM每生成一个token，都是在当前上下文状态下采取一次动作



核心目标：

$$J(\theta) = \mathbb{E}_{y \sim \pi_{\theta}(\cdot|x)}[r(x, y)]$$

策略就是语言模型：

$$\pi_{\theta}(a_t | s_t)$$

其中 x : 输入prompt; y : 模型输出; $r(x, y)$: 奖励函数

2. 主流方法：常见分类

- 基于人类反馈的强化学习 (RLHF)
- 基于 AI 反馈的强化学习 (RLAIF)
- 基于规则/程序反馈的强化学习
- 在线强化学习
- 离线强化学习 / 偏好优化
- 带 KL 约束的策略优化
- 无价值函数或简化版 RL 方法

2. 主流方法: RLHF

RLHF基本流程，三阶段：(监督微调、训练奖励模型、优化策略模型)

1. SFT

用高质量指令数据训练模型，让模型学会向助手一样回答。

- 输出：SFT模型
- 目标：

$$\max_{\theta} \sum \log \pi_{\theta}(y^*|x)$$

Tips:
 y^* : 人工编写或筛选的优质答案

2. RM (Reward Model)

对同一prompt的多个回答做偏好排序，训练奖励模型。

- 输出：奖励函数
- 常见损失函数：

$$\mathcal{L}_{RM} = -\log \sigma(r_{\phi}(x, y_w) - r_{\phi}(x, y_l))$$

Tips:
 y_w : 胜者回答, y_l : 败者回答

3. RL (Reinforce Learning)

最大化RM评分，同时用KL限制别偏离参考模型太远。

- 输出：对齐模型
- 典型目标：

$$\max_{\theta} \mathbb{E}_{y \sim \pi_{\theta}} [r_{\phi}(x, y)] - \beta \text{KL}(\pi_{\theta} || \pi_{\text{ref}})$$

Tips:
 π_{ref} : 参考模型, 通常是SFT模型

2. 主流方法：RLHF

RLHF为什么有效?

核心价值：人类未必能写出最优答案，但容易比较两个答案谁更好

预训练学到的是分布

- 最大化下一个token概率
- 擅长模仿语言模式
- 不等于“有帮助、安全、可靠”

RLHF学到的是偏好

- 让人类比较候选回答
- 训练奖励模型估计“好坏”
- 用RL优化模型行为

偏好标注 > 标准答案标注

2. 主流方法：RLHF VS RLAIIF/Constitutional AI

当反馈来自AI或规则

目的：降低人工反馈成本并让规则更可扩展

RLAIIF

- **做法：**
 1. 更强的模型作为评判者
 2. 对候选回答进行评分或排序
 3. 再训练reward model或直接做偏好优化
- **优点：**低成本、高扩展性

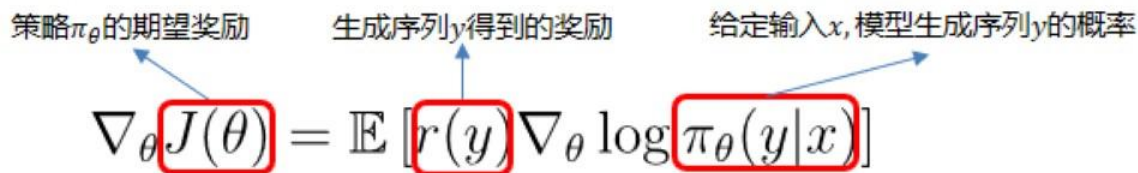
Constitutional AI

- **做法：**
 1. 提供一套“宪法原则”，如：不鼓励伤害、尊重隐私、避免歧视、在不确定时表达不确定性
 2. 让模型学习根据规则修正回答
 3. 用原则评判多个回答
 4. 再做RL或偏好优化
- **优点：**降低对人工红队的依赖、高度系统化

2. 主流方法: REINFORCE

REINFORCE是最原始的策略梯度方法, 常被用于 LLM, 尤其是在奖励明确、实现简单的任务中。

核心公式:


$$\nabla_{\theta} J(\theta) = \mathbb{E} [r(y) \nabla_{\theta} \log \pi_{\theta}(y|x)]$$

也可写成token分解:

$$\log \pi_{\theta}(y|x) = \sum_t \log \pi_{\theta}(y_t|x, y_{<t})$$

所以:

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[r(y) \sum_t \nabla_{\theta} \log \pi_{\theta}(y_t|x, y_{<t}) \right]$$

优点:

- 简单直接
- 易于接入任意奖励函数

缺点:

- 方差大
- 训练不稳定

2. 主流方法: Actor-Critic

Actor-Critic是结合了策略梯度与价值函数的双网络强化学习架构, 通过Actor负责决策选动作、Critic负责评估打分。

Critic损失函数:

当前步获得的即时奖励 折扣因子 Critic网络对状态 s_t 的价值估计

$$L(\phi) = \mathbb{E} \left[\left(\boxed{r_t} + \boxed{\gamma} V_{\phi}(s_{t+1}) - \boxed{V_{\phi}(s_t)} \right)^2 \right]$$

用TD误差更新:

$$\delta_t = r_t + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t)$$

Actor更新公式:

$$\nabla_{\theta} J(\theta) = \mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot \delta_t]$$

优点:

- 简单直接
- 易于接入任意奖励函数

缺点:

- 方差大
- 训练不稳定

2. 主流方法: KL-regularized RL

LLM强化学习中非常关键的思想不是某个具体算法，而是：更新时必须约束新策略不能偏离原模型太远。

原因：

- 保护语言能力
- 约束奖励漏洞

因此，很多LLM-RL方法都带有KL正则项：

KL散度是衡量两个概率分布之间“差异程度”的非对称指标，值越大表示两个分布越不相似，值为0表示两个分布完全相同。

$$\max_{\theta} \mathbb{E}[r(x, y)] - \beta KL(\pi_{\theta} || \pi_{ref})$$

2. 主流方法: PPO

作为RLHF中强化学习阶段的核心算法，是一种策略梯度方法，目标是在提升奖励的同时，不让策略更新幅度太大。相比REINFORCE引入优势函数和Clip截断。

经典PPO clip目标:

$$L^{CLIP}(\theta) = \mathbb{E} \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

其中: $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$

$\hat{A}_t$: 优势函数

clip: 限制单次更新幅度

PPO在LLM中通常会做一些特殊适配:

- token级策略梯度
- 价值函数
- KL惩罚

$$r = r_{RM} - \beta \sum_t \log \frac{\pi_{\theta}(a_t|s_t)}{\pi_{ref}(a_t|s_t)}$$

- response-level/token-level reward

2. 主流方法：DPO

作为RLHF的一种简化版实现方案，直接替代了传统RLHF里“训练RM和强化学习RL”的两步骤。

传统RLHF流程是：1. 训练reward model → 2. 用PPO等方法优化policy

DPO发现：在一定假设下，可以把“最优策略相对参考策略的比值”直接写成奖励差的形式，从而直接用偏好数据训练策略，无需单独训练RM和PPO。

DPO的损失形式通常是：

$$\mathcal{L}_{DPO} = -\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{ref}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{ref}(y_l|x)} \right)$$

严格来说，DPO更像一种离线偏好优化方法，而不是传统RL

2. 主流方法：DPO的扩展方法

DPO 家族的共同方向

- 1 IPO：关注过拟合与校准
- 2 KTO：从好 / 坏样本直接学习
- 3 ORPO：单阶段兼顾 SFT 与偏好
- 4 SimPO：进一步简化参考模型依赖

共同目标：降低复杂在线 RL 的成本

DPO流行之后，出现了很多变体：

这些方法的共同点是：

- 尽量避免复杂在线 RL
- 直接从偏好对中学习
- 降低训练成本

2. 主流方法：一些轻量化RL方法

由于 PPO 太“重”，近年来出现很多更轻量的方法如RLOO (REINFORCE Leave-One-Out)、GRPO (Group Relative Policy Optimization) 等

RLOO

- **做法：**

RLOO用同一prompt下多个采样回答之间的平均奖励作为baseline, 降低方差：

$$A_i = r_i - \frac{1}{K-1} \sum_{j \neq i} r_j$$

- **特点：**不需要critic、比reinforce稳定、工程上比PPO简单很多

GRPO

- **做法：**

对同一问题采样多个回答，根据组内相对表现计算优势

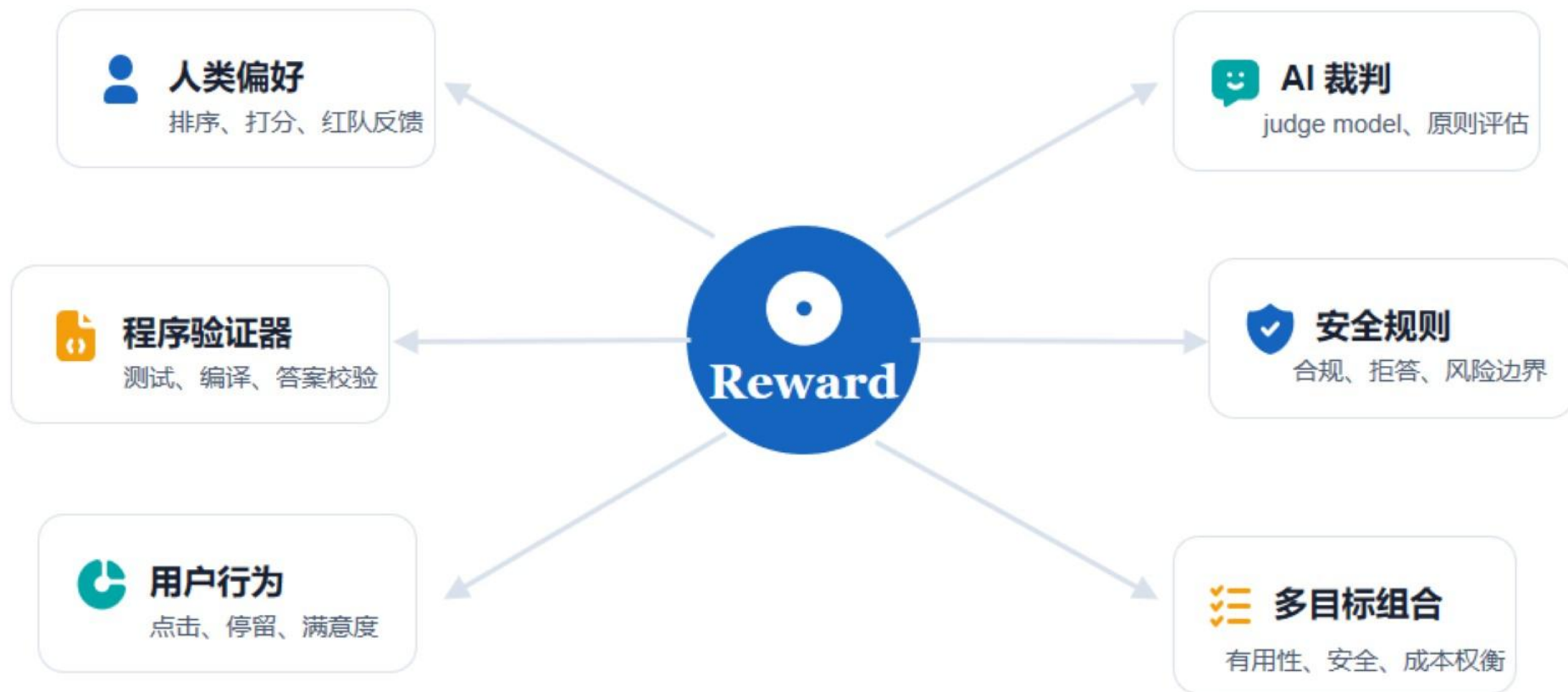
$$A_i = \frac{r_i - \mu_G}{\sigma_G}$$

- **优点：**不需要critic、特别适合可验证任务如数学/代码推理任务

3. 工程设计：方法取舍

维度	PPO	DPO	GRPO / RLOO	判断?
数据来源	在线采样 RM打分	静态偏好对 胜者 / 败者	同prompt多采样 组内比较	是否能探索
组件成本	RM+Critic+Rollout	无RM、无Critic	无Critic, 需多采样	训练预算
优势	稳定、经典、可在线	简单、便宜、易复现	适合可验证推理任务	主要卖点
局限	复杂、贵、调参敏感	依赖静态数据	依赖奖励与采样质量	风险来源
适合场景	经典对齐训练	工业低成本迭代	数学、代码、推理	选型结论

3. 工程设计：奖励设计



3. 工程设计：失败模式与训练技巧



奖励模型会被优化，也会被钻空子

模型不会理解“真实目标”，它只会沿着奖励信号找高分路径。

Reward hacking

策略过度优化奖励模型

奖励模型在训练分布有效，但对新分布评分不可靠

偏好更长、更自信的回答

过度迎合用户/裁判



KL约束，防止偏离reference太远

奖励/优势归一化

课程学习*

采样控制(温度、Top-p)

混合目标：SFT、RL、KL

*: Curriculum learning, 指逐步增加难度

4. 发展趋势

从“重PPO”走向轻量偏好优化

DPO、ORPO、SimPO等方法更容易训练和复现。

从人类反馈走向混合反馈

人类、AI、程序验证器和用户行为会联合使用。

从结果奖励走向过程奖励

PRM更适合数学、代码和长链推理优化。

从单轮文本生成走向Agent RL

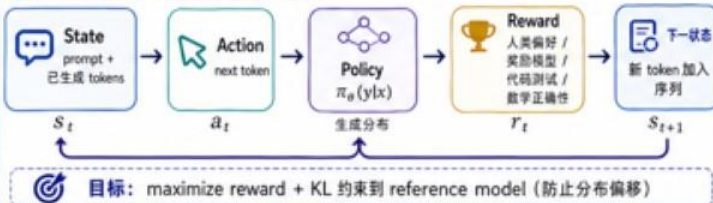
工具调用、多步规划和环境交互让RL更接近经典设定。

从对齐走向能力提升

推理、代码、任务成功率成为RL的重要目标。

5. 课程总结

1 基本框架：序列决策过程



2 经典 RLHF: 三阶段流程



5 其他常见方法



3 DPO 系方法：直接用偏好数据，简化优化流程



4 在线强化学习 / 推理增强



右下：方法对比一览

方法	是否需要 reward model	是否需要 value model	实现复杂度	更新模型	适用场景 / 特点
PPO	需要	需要	高 ★★★★★	✓	经典强大, 灵活性高, 适合通用场景
DPO	不需要	不需要	低 ★★★★★	✓	简单稳定, 训练高效, 广泛适用
GRPO	需要 (或可用显式奖励)	不需要	中等 ★★★★★	✓	无 value model, 适合数学/代码/推理任务
Rejection Sampling	需要 (打分器)	不需要	很低 ★★★★★	✗	便宜快捷, 提升输出质量, 但不更新模型
RLAIF	需要 (AI Judge)	取决于方法	中等 ★★★★★	✓	降低人工成本, 可规模化构建偏好数据

6 关键问题

